Processamento Estatístico de Sinais

Detecção de Alvo em Clutter via Modelagem AR e Filtro Branqueador

Projeto 3

1 Contexto

Em um sistema de radar (ou sonar), o sinal recebido é a soma de três componentes:

$$x(n) = s(n) + c(n) + v(n) \tag{1}$$

onde:

- $s(n) = A_s \sin(n\omega_{\text{alvo}} + \phi_s)$ é o retorno de um **alvo** – uma senoide de fase aleatória, fraca e de frequência desconhecida (um *processo harmônico*);
- $c(n)$ é o **clutter** – reflexos do ambiente (terreno, mar, chuva), um processo WSS de banda estreita, tipicamente muito mais forte que o alvo;
- $v(n)$ é **ruído branco** do receptor, de potência menor que o clutter.

O clutter tem *ressonâncias espectrais* bem definidas (energia concentrada em faixas estreitas de frequência), enquanto o alvo é um tom puro (um impulso espectral isolado, sem banda nenhuma). Fisicamente, esse caráter de banda estreita do clutter vem de ele ser a superposição de muitos ecos individuais, moldados pela resposta em frequência do meio de propagação (relevo, superfície do mar, chuva) e do próprio receptor – exatamente o tipo de "ruído branco filtrado por um sistema linear"que a decomposição de Wold garante ser possível para qualquer processo WSS regular. Por isso um modelo **AR** (que é, por construção, ruído branco através de um filtro all-pole) é uma escolha natural para representar o clutter, mesmo sem conhecer a priori sua ordem ou seus polos exatos – é a mesma lógica de modelagem paramétrica de espectro vista no capítulo, agora aplicada a um sinal que você não construiu.

O objetivo desta atividade é: (1) estimar um modelo AR do clutter diretamente de $x(n)$, sem conhecer seus parâmetros verdadeiros; (2) usar esse modelo para construir um **filtro branqueador** e revelar o alvo, escondido sob um clutter muito mais forte; e (3) entender **precisamente** o que esse branqueamento melhora na detecção – e o que ele *não* melhora, um resultado teórico menos óbvio do que parece à primeira vista.

2 Parâmetros do problema

Gere os dados sintéticos com os seguintes parâmetros, para que seus resultados sejam comparáveis aos do gabarito:

O clutter é gerado filtrando ruído branco de variância σ_c^2 por um filtro AR(4) com os polos acima (use `poly` para obter os coeficientes $A_p(z)$ a partir dos polos). A potência do alvo definida acima o torna **genuinamente fraco** frente ao clutter local – não deve ser visível a olho nu no domínio do tempo.

Parâmetro	Valor
Polos do clutter (AR(4))	$0{,}92\,e^{\pm j0{,}15\pi}$ e $0{,}85\,e^{\pm j0{,}42\pi}$
Variância do ruído branco do clutter, σ_c^2	1
Frequência do alvo, ω_{alvo}	$0{,}40\pi$ rad/amostra
Potência do alvo	$P_c(\omega_{\text{alvo}})/150$, onde P_c é a densidade espectral teórica do clutter
Variância do ruído branco do receptor, σ_v^2	0,3
Número de amostras, N	16384
Fase do alvo, ϕ_s	aleatória, uniforme em $(-\pi, \pi)$, sorteada a cada realização

3 Tarefas

1. **Periodograma bruto.** Estime o espectro de potência de $x(n)$ pelo método de Welch (blocos com janela, com média entre eles) e mostre que o clutter domina o espectro – o alvo não deve se distinguir claramente do clutter nesta etapa.

```
Nfft = 1024;
win = hamming(Nfft)';          % qualquer janela reduz vazamento
    espectral
winnorm = sum(win.^2)/Nfft;  % normalizacao de potencia da janela
nblocks = floor(N/Nfft);

Pxx = zeros(1,Nfft);
for b = 1:nblocks
    seg = x((b-1)*Nfft+1 : b*Nfft)';
    Xb = fft(seg.*win);
    Pxx = Pxx + abs(Xb).^2/(Nfft*winnorm);
end
Pxx = Pxx/nblocks;    % periodograma de Welch (media entre blocos)
```

> **Dica**
>
> Use exatamente esta mesma rotina (mesmo `Nfft`, mesma janela) para calcular o periodograma **antes e depois** do branqueamento (Tarefas 3 e seguintes) – só assim as comparações de razão pico/piso e de fração do espectro acima de um limiar são diretas, sem um fator de normalização diferente escondido entre as duas curvas.

2. **Seleção da ordem do modelo AR.** Na prática não sabemos que o clutter é AR(4). Estime $r_x(0), \ldots, r_x(2p_{\max})$ a partir do próprio $x(n)$ (por exemplo $p_{\max} = 10$), resolva Yule-Walker *padrão* para cada ordem $p = 1, \ldots, p_{\max}$ (matriz de Toeplitz com $r_x(0), \ldots, r_x(p-1)$), e observe como o erro de predição $\hat{\sigma}_v^2(p)$ cai com p. Escolha a ordem pelo "cotovelo" dessa curva – por exemplo, a primeira ordem cuja queda relativa para a ordem seguinte já fica abaixo de um limiar pequeno (ex. 5%).

Atenção

$x(n)$ tem ruído branco do receptor somado, que contamina *apenas* $r_x(0)$ (ruído branco aditivo não afeta $r_x(k)$ para $k \neq 0$). Como as ressonâncias do clutter são estreitas (polos perto do círculo unitário), o sistema de Yule-Walker **padrão** (que usa $r_x(0)$) fica mal-condicionado, e esse pequeno viés em $r_x(0)$ pode se propagar como um erro *grande* nos coeficientes estimados – mas só na etapa *seguinte* (o ajuste final do filtro branqueador, Tarefa 3). Para a seleção de ordem em si, o Yule-Walker padrão é suficiente: o que importa aqui é só o *formato* da curva de erro de predição (onde fica o cotovelo), não os coeficientes exatos.

Dica

É perfeitamente possível que o cotovelo indique uma ordem **menor** que a ordem verdadeira do clutter (4 neste caso) – por exemplo, se a queda relativa de erro entre duas ordens baixas já for pequena por acaso, mesmo havendo uma queda maior mais adiante. Isso não invalida o exercício: um modelo AR aproximado (ordem errada, mas capturando ao menos a ressonância dominante) já costuma ser suficiente para o ganho de detecção que você vai quantificar nas próximas tarefas. Reporte a ordem que seu critério selecionou e siga em frente com ela – não ajuste manualmente até "acertar"p=4.

3. **Filtro branqueador.** Com a ordem p escolhida na tarefa anterior, resolva agora o Yule-Walker **modificado** (usando só defasagens $k = p+1, \ldots, 2p$, nunca $r_x(0)$, pelo motivo explicado no quadro de atenção acima):

$$r_x(k) + \sum_{l=1}^{p} \hat{a}_p(l)\, r_x(k-l) = 0, \qquad k = p+1, \ldots, 2p \tag{2}$$

```
p = p_selected;                    % ordem escolhida na Tarefa anterior
kidx = (p+1):(2*p);                % defasagens k = p+1,...,2p -- nunca k=0
Mmod = zeros(p); rhsmod = zeros(p,1);
for i = 1:p
    kk = kidx(i);
    for l = 1:p
        Mmod(i,l) = ???;           % <-- r_x na defasagem correspondente (ver equacao acima)
    end
    rhsmod(i) = -rx_hat(kk+1);
end
a_hat = (Mmod\rhsmod)';
a_hat_full = [1, a_hat];           % A_p(z) = 1 + a_hat(1)z^-1 + ... + a_hat(p)z^-p

e = filter(a_hat_full, 1, x);      % sinal branqueado
```

O polinômio $\hat{A}_p(z)$ é o **filtro branqueador**: aplicado a $x(n)$, a parte de clutter deveria virar aproximadamente ruído branco. Recalcule o periodograma de $e(n)$ com a mesma rotina da Tarefa 1 e compare visualmente com o periodograma de $x(n)$.

Dica

Se `rx_hat` na Tarefa 2 só foi calculado até a defasagem $2p_{\max}$, e sua ordem escolhida p satisfaz $2p \leq 2p_{\max}$, você já tem tudo que precisa – não é necessário recalcular a autocorrelação.

4. **Pergunta teórica: o branqueamento ajuda um detector CFAR local?** Antes de medir o ganho de detecção "global"(próxima tarefa), investigue algo mais sutil. Suponha que você já soubesse, a priori, a frequência exata do alvo, e comparasse o pico ali com o clutter bem ao lado – uma **célula de referência local**, como faz um detector CFAR (*Constant False Alarm Rate*). Nesse cenário, o branqueamento ajuda?

Atenção

A resposta teórica é **não**: filtrar por $H(z)$ multiplica tanto o sinal quanto o clutter pelo *mesmo* $|H(e^{j\omega})|^2$, na *mesma* frequência (resultado de filtragem de processos WSS visto no capítulo) – a razão pico/piso **local** é invariante ao filtro. Não existe "almoço grátis"local: o branqueamento não cria informação nova sobre o alvo naquela célula especificamente, só redistribui energia entre frequências diferentes.

Verifique isso numericamente: compare a razão entre o valor do periodograma na frequência do alvo e o valor típico (mediana) de um conjunto de células de referência *vizinhas*, antes e depois do branqueamento. A razão deve mudar pouco (poucos dB), bem menos do que a mudança dramática que você vai observar na próxima tarefa.

Dica

Ao escolher as células de referência, exclua algumas células de **guarda** bem ao redor do próprio pico do alvo – a energia do pico "vaza"um pouco para as células vizinhas mais próximas (vazamento espectral da janela), e incluí-las na referência enviesaria sua estimativa do piso de clutter local para cima. Como a grade de frequência é circular (o último bin da FFT é vizinho do primeiro), use `mod` para os índices:

```
[~, bin_alvo] = min(abs(wsorted - omega_alvo_true));  % indice do alvo na grade ordenada
guarda = 3; janela = 40;                % celulas de guarda e de referencia (ajuste se quiser)
idx_janela = mod((bin_alvo-janela : bin_alvo+janela) - 1, Nfft) + 1;
idx_guarda = mod((bin_alvo-guarda : bin_alvo+guarda) - 1, Nfft) + 1;
idx_ref = setdiff(idx_janela, idx_guarda);   % referencia = janela MENOS as celulas de guarda
```

5. **Quantificando o ganho de detecção com um limiar global.** Agora considere o cenário oposto ao da tarefa anterior: você *não* sabe a priori onde procurar o alvo, e usa um único limiar **fixo** para o espectro inteiro (sem adaptar célula a célula). Quantos % do espectro cruzam esse limiar antes e depois do branqueamento? O que isso implica para a taxa de falsos alarmes de um detector simples que não sabe onde olhar?

Dica

Diferente da Tarefa 4, aqui o branqueamento deve fazer uma diferença enorme – não porque o alvo fique mais forte (ele não fica, pela mesma invariância local da tarefa anterior), mas porque o clutter deixa de ocupar uma fatia larga do espectro acima de qualquer limiar razoável. O alvo continua cruzando o limiar antes e depois; o que muda é que ele deixa de ter *concorrência* de falsos alarmes vindos do clutter.

6. **Estimando a frequência do alvo.** Uma vez que o alvo esteja identificável no espectro branqueado, estime ω_{alvo} pelo pico do periodograma de $e(n)$ e compare com o valor verdadeiro

usado na geração dos dados.

7. **Validação por Monte Carlo.** Repita os passos acima (gerar dados, branquear, medir a razão CFAR local e a fração do espectro acima do limiar global, estimar a frequência) em pelo menos 30 realizações independentes – mesmos parâmetros, ruído e fase do alvo diferentes a cada vez – e reporte a taxa de sucesso e o erro médio de estimação de frequência.

Dica

Use a **mesma** ordem p (escolhida uma única vez na Tarefa 2) em todas as 30 realizações – não re-selecione a ordem a cada realização. O objetivo aqui é testar a *robustez* do resultado obtido, não repetir a etapa de seleção de ordem 30 vezes.

4 Funções úteis do MATLAB

Função	Uso nesta atividade
`poly(raizes)`	Constrói os coeficientes de um polinômio a partir de suas raízes – usada para obter $A_p(z)$ do clutter a partir dos polos dados na tabela de parâmetros.
`filter(b,a,x)`	Aplica um filtro IIR/FIR a um sinal – usada para gerar o clutter AR (numerador 1) e para aplicar o filtro branqueador (denominador 1, numerador $\hat{A}_p(z)$).
`hamming(N)`	Janela para reduzir vazamento espectral antes de qualquer análise em um bloco finito (método de Welch).
`fft(x)`	Transformada discreta de Fourier – núcleo do cálculo do periodograma de cada bloco.
`toeplitz(r)`	Constrói a matriz de autocorrelação (simétrica, com r na primeira linha/coluna) usada no Yule-Walker **padrão** da seleção de ordem.
`arrayfun(fun,vetor)`	Aplica uma função a cada elemento de um vetor, devolvendo um vetor de resultados – usada para calcular $\hat{r}_x(k)$ para vários valores de k de uma vez.
`mod(x,n)`	Resto da divisão – usada para tratar a grade de frequência como circular ao definir células de guarda/referência perto das bordas da FFT.
`setdiff(A,B)`	Elementos de `A` que não estão em `B` – usada para obter as células de referência (janela menos as células de guarda).
`median(x)`	Mediana – usada como estimativa robusta do piso de clutter (local e global), pouco sensível a um ou outro valor atípico.
`histogram(x,nbins)`	Histograma – usada para visualizar a distribuição do erro de frequência nas realizações de Monte Carlo.
`xline(valor,...)`	Desenha uma linha vertical de referência num gráfico – usada para marcar a frequência verdadeira do alvo sobre o periodograma.

5 Entregáveis

- Código comentado (MATLAB) que reproduza todas as etapas acima.
- Gráficos: periodograma antes e depois do branqueamento; curva do erro de predição vs. ordem do modelo AR.

- Tabela com: ordem escolhida, razão CFAR local antes/depois (Tarefa 4), % do espectro acima do limiar global antes/depois (Tarefa 5), frequência estimada vs. verdadeira, resultados do Monte Carlo.
- Um parágrafo respondendo: *Qual foi o efeito do filtro branqueador sobre o sinal aleatório? Por que ele ajuda o detector de limiar global mas não o detector CFAR local?*

6 Critérios de avaliação

- Correção da modelagem AR e da seleção de ordem (20%)
- Implementação correta do filtro branqueador (Yule-Walker modificado) (20%)
- Verificação numérica da invariância do CFAR local (15%)
- Quantificação do ganho com limiar global (15%)
- Estimação de frequência e validação por Monte Carlo (15%)
- Discussão teórica (por que um resultado melhora e o outro não) (15%)